

Abstract

The study looks at classifying images of dogs & cats. It uses three kinds of models: Fully Connected Neural Network (FC), Convolutional Neural Network (CNN), and Transfer Learning. The team worked with a dataset that has 25,000 images. These were split into test, validation, & training sets.

The main goals? They wanted to find out which models work best, check how dataset augmentation helps, and look into hyperparameters. All models did pretty well, but CNN & Transfer Learning did a bit better than FCNN.

For evaluation, they used different methods: accuracy, loss curves, ROC-AUC curves, as well as confusion matrices. Model generalization was also an important factor in the study. Plus, they pointed out some areas where improvements could be made in the future.

Table of Contents

1. Introduction To Image Classification.....	1
1.1. About Datasets.....	1
1.2. Data Understanding.....	3
2. Methodology.....	3
2.1. Image Classification with Fully Connected Neural Network.....	3
2.1.1. Model Summary.....	3
2.1.2. Training of The Model.....	5
2.1.3. Evaluation Metrics.....	6
2.1.4. Predictions.....	8
2.2. Image Classification with Convolutional Neural Network.....	9
2.2.1. Model Summary.....	9
2.2.2. Training of The Model.....	9
2.2.3. Evaluation Metrics.....	11
2.2.4. Predictions.....	12
2.3. Image Classification with Transfer Learning.....	13
2.3.1. Model Summary.....	13
2.3.2. Training of The Model.....	14
2.3.3. Evaluation Metrics.....	16
3. Findings & Discussions.....	18
References.....	19

1. Introduction To Image Classification

Image classification is a process in which the computer learns with the help of image contains. In this case, it may be able to say whether a given picture is of cat or dog after training with many images containing cats and dogs.

1.1. About Datasets

A Datasets filled with twenty-five thousand (25,000) pictures are of dogs and cats, which are separated into Train (20,000) and Test (5,000) folders for algorithm training and validation. There will be descriptions about what a Neural Network is; why we need it? How does it work in the case of image recognition? Three distinct models of artificial neural networks will also be elaborated upon: Fully Connected Neural Network (FCNN), Convolutional Neural Network (CNN), and Transfer Learning. (Anon., n.d.)

```
Class counts in validation dataset:  
Class Cats: 2500 images  
Class Dogs: 2500 images  
Total number of files in test dataset: 5000
```

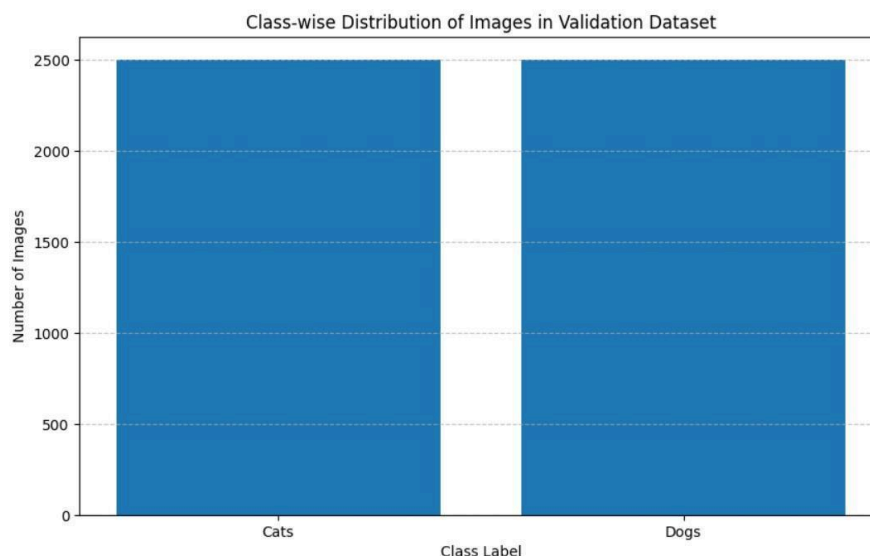


Figure 1: Validation dataset

Class counts in train dataset:
Class Cats: 10000 images
Class Dogs: 10000 images
Total number of files in Train dataset: 20000



Figure 2: Train dataset

Class counts in Test dataset:
Class Dogs: 1213 images
Class Cats: 1227 images
Total number of files in Test dataset: 2440

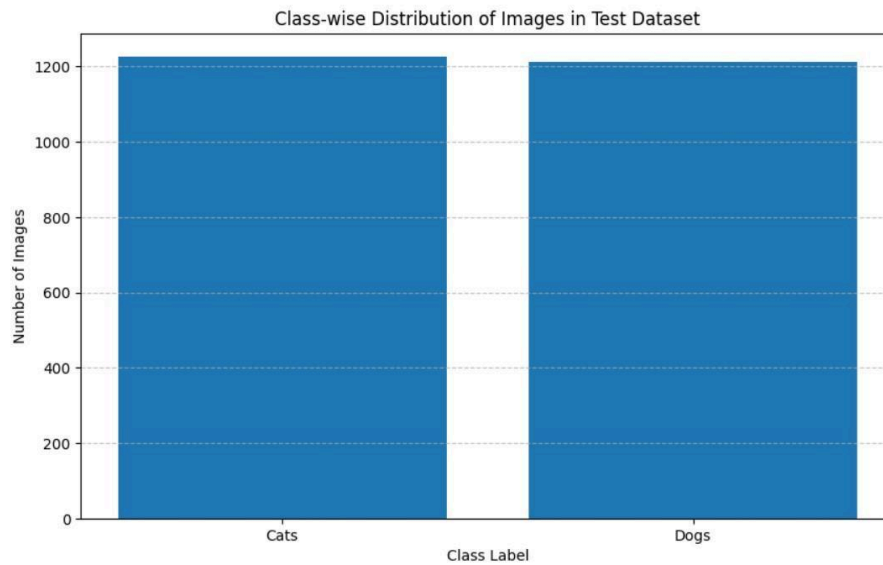


Figure 3: Test Dataset

1.2. Data Understanding

We made an attempt to make sure that we had a balanced distribution of images within the datasets which included 20000 photos in the train set, 5000 in the validation set and 2440 in the test set. In order to diversify our training data using TensorFlow data generator, we also employed various augmentation methods such as brightness adjustment. Then, we trained, validated and tested on the datasets accordingly, each of which was designed for one particular part of the classification task. A Keras' `image_dataset_from_directory` function powered TensorFlow data generator has been created in order to handle the dataset effectively. Such a generator enabled us employ data augmentation techniques like brightness adjustment that created some variability into it and thus increased its generalizability. Consequently, these dataset underwent rigorous preparation for training, validation and testing phases with each assigned to a specific model for complete examination of all aspects. (Bang Liu, n.d.)

2. Methodology

For Image classification between cat and dog, We used three model: Fully Connected (Bang Liu, n.d.) (Daniele Corradetti, n.d.)

2.1. Image Classification with Fully Connected Neural Network

2.1.1. Model Summary

Fully Connected Neural Network (FCNN) model with 14 layers: it commences with "Flatten" to reshape the input data, followed by twelve ReLU activated units Dense layers (each containing 128 neurons), and ends up with one unit Dense output layer for binary classification using sigmoid activation. Adam optimizer, binary cross-entropy loss function and accuracy as metric are used in compiling this model. After initializing the model is trained and evaluated using graphs for accuracy and loss on training and validation sets then, its performance is further assessed through confusion matrix and test data.

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 196608)	0
dense (Dense)	(None, 128)	25165952
dense_1 (Dense)	(None, 128)	16512
dense_2 (Dense)	(None, 128)	16512
dense_3 (Dense)	(None, 128)	16512
dense_4 (Dense)	(None, 128)	16512
dense_5 (Dense)	(None, 128)	16512
dense_6 (Dense)	(None, 128)	16512
dense_7 (Dense)	(None, 128)	16512
dense_8 (Dense)	(None, 128)	16512
dense_9 (Dense)	(None, 128)	16512
dense_10 (Dense)	(None, 128)	16512
dense_11 (Dense)	(None, 128)	16512
dense_12 (Dense)	(None, 128)	16512
dense_13 (Dense)	(None, 1)	129
Total params: 25364225 (96.76 MB)		
Trainable params: 25364225 (96.76 MB)		
Non-trainable params: 0 (0.00 Byte)		

Figure 4: FCNN Model

2.1.2. Training of The Model

The model uses binary cross-entropy as the loss function to measure how well it predicts two classes, with output probabilities ranging from 0 to 1. It employs the Adam optimizer, which adjusts the learning rate during training to improve performance. The model is trained for 10 epochs, meaning it goes through the dataset multiple times to fine-tune its parameters. The training and validation loss curves help visualize how the model is learning, with one line showing accuracy on the training set and another on the validation set, providing insight into its performance over time.

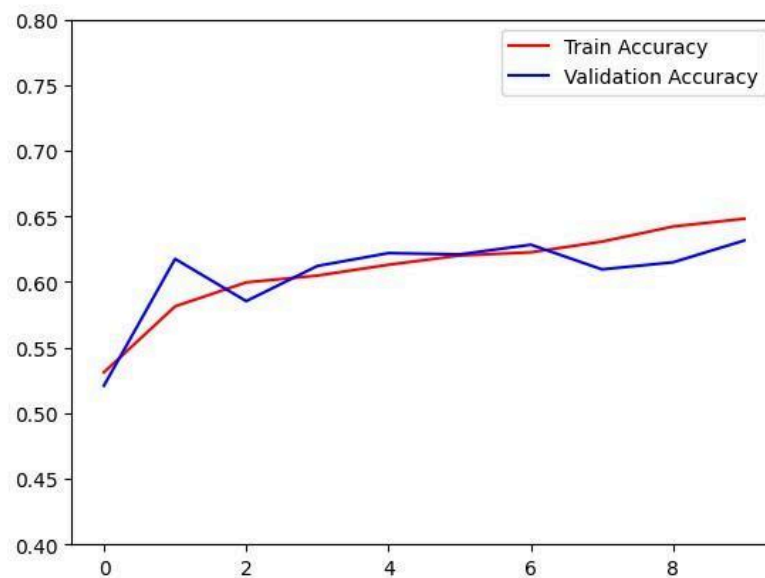


Figure 5: Train Accuracy vs Validation Accuracy

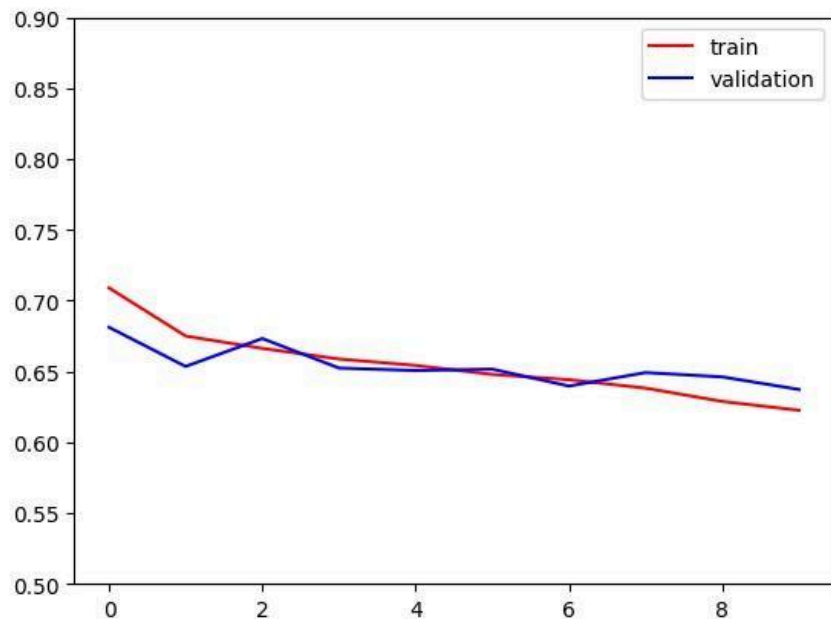


Figure 6: Train Loss vs Validation loss

2.1.3. Evaluation Metrics

After regularization, the model's training and validation accuracies stabilized after initially varying. As training went on, both accuracies closely corresponded, suggesting balanced performance. A thorough investigation that included ROC-AUC and a confusion matrix verified the continuously excellent accuracy.

```
Epoch 10/10
313/313 [=====] - 49s 154ms/step - loss: 0.6226 - accuracy: 0.6482 - val_loss: 0.6373 - val_accuracy: 0.6316
```

Figure 7: Model Accuracy of FCNN

```
Number of images with correct predictions: 3158
Number of images with incorrect predictions: 1842
```

Figure 8 : Correct Predication vs incorrect predication

www.elsevier.com/locate/jmb

1

2.1.4. Predictions

Test data was used to gauge the model's performance and determine how accurate the forecasts were. Although the total accuracy was poor, a confusion matrix revealed that more than 60% of predictions were accurate, suggesting stability and neutrality.

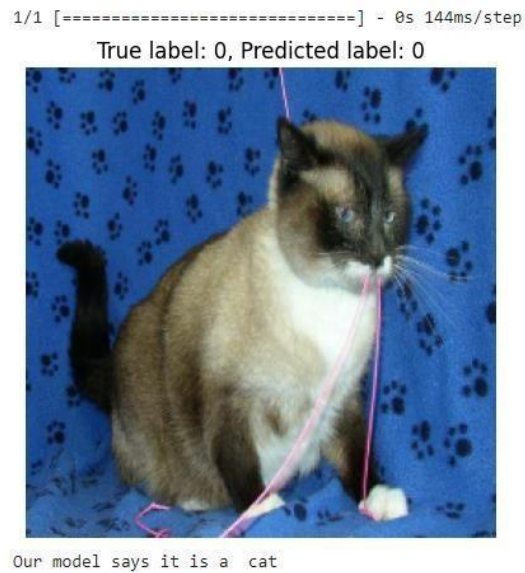


Figure 12: Predication of the model

2.2. Image Classification with Convolutional Neural Network

2.2.1. Model Summary

The model has 14 layers in total. It's built for binary classification. First, it uses a Flatten that processes visual data. Then come twelve Dense layers, each featuring 128 units that use ReLU activation. After that, there's a final Dense layer. This one has just a single unit, and it applies a sigmoid activation function.

For optimization, this model uses the Adam optimizer. The loss function is Binary Cross Entropy. Accuracy is the metric that measures performance.

The code provided does quite a bit! It sets up, compiles, & trains the model using a dataset. Then, it checks how well the model performs on both training and validation sets. A confusion matrix is created to visualize results. Plus, the model gets saved for later use. Finally, it tests how well it performs on new data. (Bang Liu, n.d.)

2.2.2. Training of The Model

The model picked binary cross-entropy as its loss function. This one great for tasks with two classes. helps measure probabilities. To boost performance & things up, the Adam optimizer was. It's popular because of its adjustable learning rates and momentum. The training wrapped up after 15 epochs.

When looking at the training & validation loss curves, we can see that training loss goes down, which means learning is happening. But wait! There's a possible rise in validation loss which might hint at overfitting. This difference signals a need for regularization techniques to help with generalization.

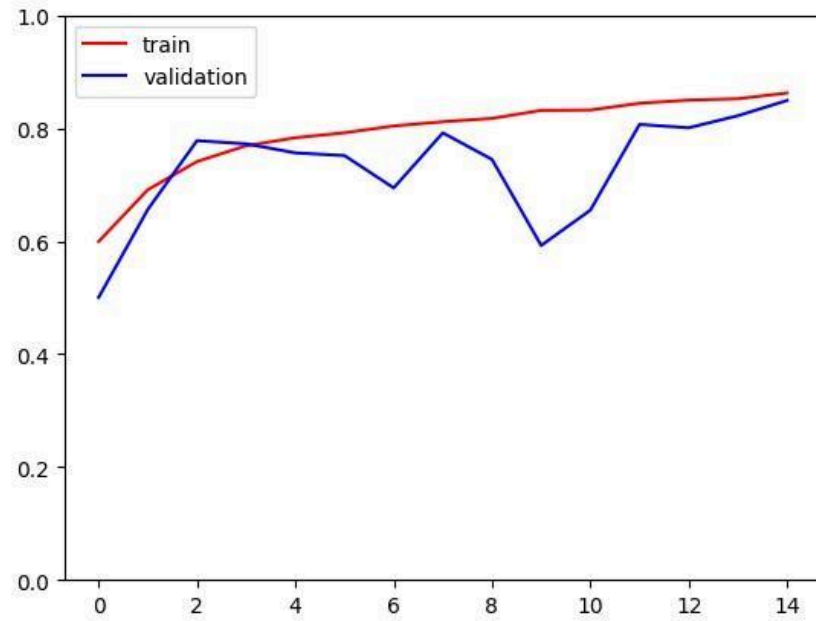


Figure 13: Train accuracy vs Validation Accuracy

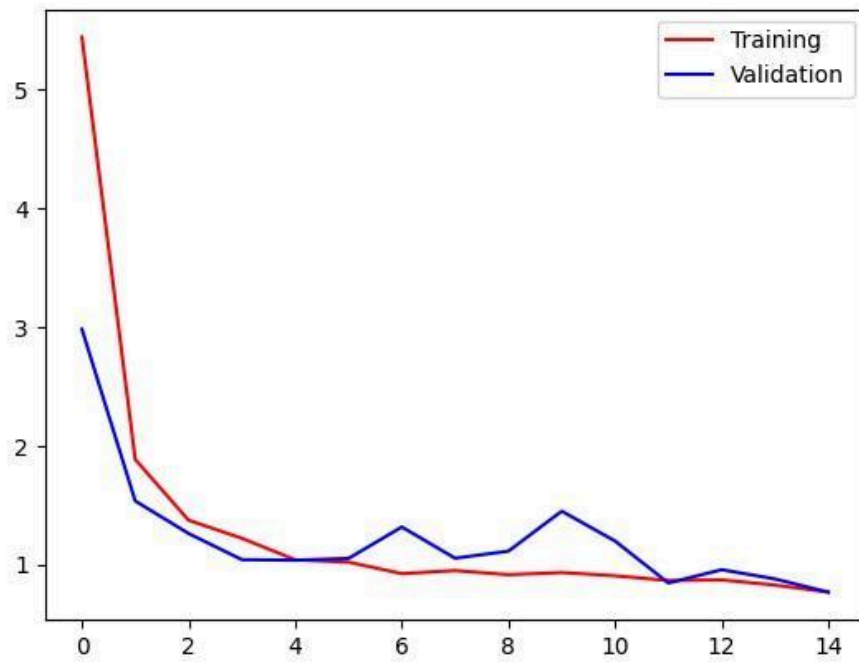


Figure 14: Train Loss vs Validation loss

2.2.3. Evaluation Metrics

The pattern shows that validation loss is often higher than training loss. This suggests there's a chance overfitting happening. I could try simplifying the model. Another idea is to add more data to the training set. Keeping an eye on accuracy can give you helpful clues to boost how well the model works.

```
Number of images with correct predictions: 4246
Number of images with incorrect predictions: 754
```

Figure 15: CNN MODEL incorrect and correct predictions

```
Number of images with correct predictions: 17461
Number of images with incorrect predictions: 2539
```

Figure 16: CNN MODEL incorrect and correct predictions

```
Number of images with correct predictions: 2122
Number of images with incorrect predictions: 374
```

Figure 17: CNN MODEL incorrect and correct predictions

```
Epoch 15/15
313/313 [=====] - 68s 216ms/step - loss: 0.7668 - accuracy: 0.8624 - val_loss: 0.7653 - val_accuracy: 0.8492
```

Figure 18: CNN MODEL ACCURACY

```
Accuracy: 0.8504098360655737
Precision: 0.8364524003254679
Recall: 0.8624161073825504
F1 Score: 0.849235852953325
```

Figure 19: Accuracy and Precision

2.2.4. Predictions

To check how the model handles new data, we made some guesses on a random selection of test images. To see how accurate those guesses were, we compared them with the actual labels. A confusion matrix helped with this. It broke down true positives, false positives, true negatives, and false negatives, giving a full picture of how well the model did. (Daniele Corradetti, n.d.)

```
1/1 [=====] - 0s 18ms/step
```

True label: 0, Predicted label: 0



Our model says it is a cat

Figure 20: CNN MODEL prediction

2.3. Image Classification with Transfer Learning

2.3.1. Model Summary

The model makes good use of a pre-trained convolutional base. This helps with efficient feature extraction. On top of that, there are Dense layers for extra. We've got two Dense layers that each have 256 units using ReLU activation. For the very last layer, a sigmoid function is for binary classification. When it comes to training, the RMSprop optimizer does its job with a learning rate set at $1e-5$. This makes for smooth adjustments during the training process. Accuracy is what matters most here, & the loss function is binary cross-entropy to help achieve precise predictions.

Model: "vgg16"

Layer (type)	Output Shape	Param #
input_1 (InputLayer)	[(None, 256, 256, 3)]	0
block1_conv1 (Conv2D)	(None, 256, 256, 64)	1792
block1_conv2 (Conv2D)	(None, 256, 256, 64)	36928
block1_pool (MaxPooling2D)	(None, 128, 128, 64)	0
block2_conv1 (Conv2D)	(None, 128, 128, 128)	73856
block2_conv2 (Conv2D)	(None, 128, 128, 128)	147584
block2_pool (MaxPooling2D)	(None, 64, 64, 128)	0
block3_conv1 (Conv2D)	(None, 64, 64, 256)	295168
block3_conv2 (Conv2D)	(None, 64, 64, 256)	590080
block3_conv3 (Conv2D)	(None, 64, 64, 256)	590080
block3_pool (MaxPooling2D)	(None, 32, 32, 256)	0
block4_conv1 (Conv2D)	(None, 32, 32, 512)	1180160
block4_conv2 (Conv2D)	(None, 32, 32, 512)	2359808
block4_conv3 (Conv2D)	(None, 32, 32, 512)	2359808
block4_pool (MaxPooling2D)	(None, 16, 16, 512)	0
block5_conv1 (Conv2D)	(None, 16, 16, 512)	2359808
block5_conv2 (Conv2D)	(None, 16, 16, 512)	2359808
block5_conv3 (Conv2D)	(None, 16, 16, 512)	2359808
block5_pool (MaxPooling2D)	(None, 8, 8, 512)	0

=====

Total params: 14714688 (56.13 MB)
Trainable params: 7079424 (27.01 MB)
Non-trainable params: 7635264 (29.13 MB)

Figure 21: transfer learning model summary

2.3.2. Training of The Model

Fine-tuning technique: the model tweaks layers above that pre-trained base but uses a lower learning rate. Why? To adapt high-level features for specific tasks while keeping what it learned from the pre-trained base. It's important to watch training dynamics too. Comparing the training & validation loss curves over different epochs helps us see how well it learns. This way we can spot any overfitting or underfitting issues and decide if we should change the training strategy or even tweak the model's architecture as needed.

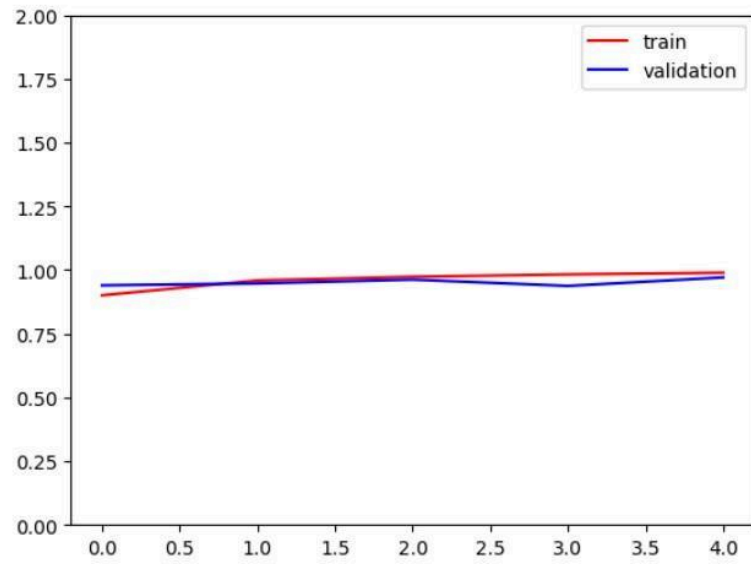


Figure 22: Transfer Learning

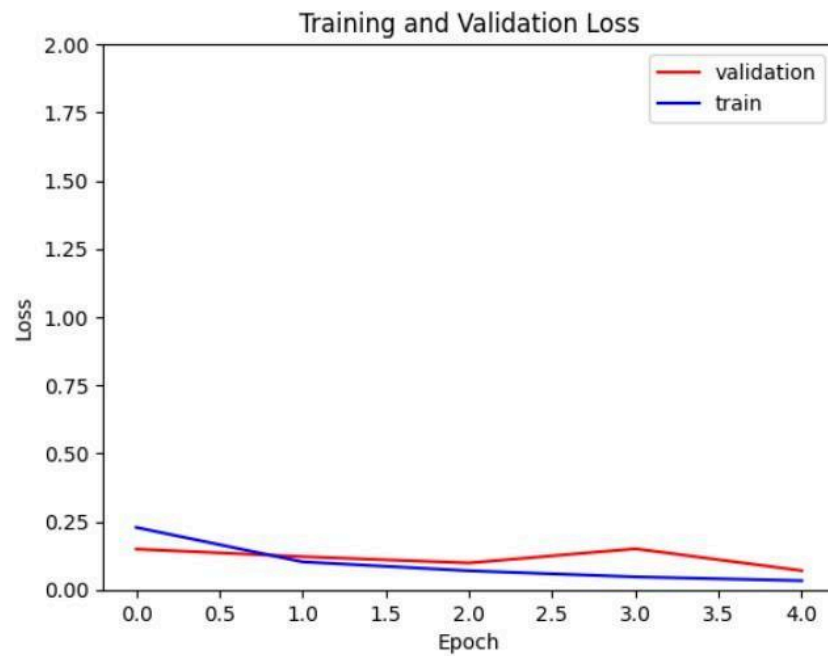


Figure 23: Training and Validation loss

2.3.3. Evaluation Metrics

The model's performance got checked using three main measures: accuracy, the confusion matrix, & loss. Accuracy shows how many correct predictions there were compared to how many total predictions were made on the validation dataset. The confusion matrix gives a clear picture of how well the model performed. It includes details like true positives, false positives, true negatives, & false negatives. (Daniele Corradetti, n.d.)

```
Number of images with correct predictions: 4856  
Number of images with incorrect predictions: 144
```

Figure 24: Correct and incorrect prediction

```
Epoch 5/5  
313/313 [=====] - 149s 473ms/step - loss: 0.0332 - accuracy: 0.9895 - val_loss: 0.0700 - val_accuracy: 0.9712
```

Figure 25: model accuracy

```
Number of images with correct predictions: 19950  
Number of images with incorrect predictions: 50
```

Figure 26: Correct and incorrect prediction of train

```
Number of images with correct predictions: 2371  
Number of images with incorrect predictions: 69
```

Figure 27: Correct and incorrect prediction of test

```
Accuracy: 0.9725409836065574  
Precision: 0.9718423169750603  
Recall: 0.9741935483870968  
F1 Score: 0.9730165122835279
```

2.3.4. Prediction

To test the model, it classified pictures picked at random from the test dataset into two groups: dogs or cats. Then, we compared the predicted labels with the real labels. Every image came with its matching labels so we could see how it did.

1/1 [=====] - 1s 942ms/step

True label: 0, Predicted label: 0



Our model says it is a cat

Figure 28: Prediction model of the transfer learning

3. Findings & Discussions

The study titled "Dog vs. Cat Image Classification Using Neural Networks" looks into how different methods of neural networks can help identify images of dogs & cats. It takes a close look at Fully Connected Neural Networks (FCNN), Convolutional Neural Networks (CNN), along with Transfer Learning techniques. The researchers used a dataset filled with 25,000

images. dataset was divided into training, validation, & test sets. The goal of the research? To figure out which models worked the best and what factors influenced their performance.

FCNN and CNN models had accuracy rates between 60% to 80%. That sounds pretty good! Still, they struggled with stability. The evaluation ways included measuring accuracy, looking at loss curves, analyzing ROC-AUC curves, and using confusion matrices for a clearer picture of how well the models worked. Interestingly, the ROC-AUC curve showed that models with transfer learning did better every time compared to FCNN and CNN models in the entire dataset. This suggests that using transfer learning is a smarter choice for this kind of image classification task.

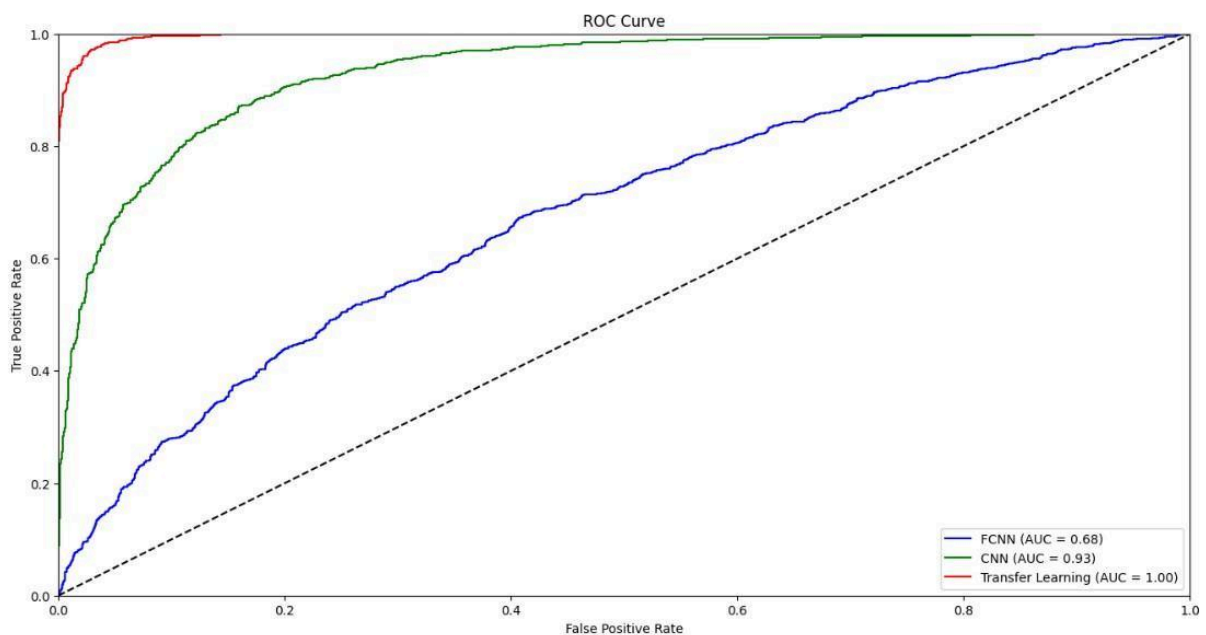


Figure 29: ROC Curve

References

Anon., n.d. *Kaggle DogVCat*. [Online]
Available at: <https://www.kaggle.com/datasets/salader/dogs-vs-cats>
[Accessed 2024].

Bang Liu, Y. L. K. Z., n.d. *DogCat_report.pdf*. [Online]
Available at: https://www-labs.iro.umontreal.ca/~liubang/files/DogCat_report.pdf
[Accessed 2024].